

■ Project 1: "TerraSpectra" — Hyperspectral Crop Disease Forecasting

Ultimate Master Plan & Week-1 Implementation Guide

■ SECTION 1: Full Project Description & Specification (From Infotact Document Volume II)

■ Project Title

Project 1 — "TerraSpectra": Hyperspectral Crop Disease Forecasting

■ Domain & Specialization

Precision Agriculture & Advanced Computer Vision

■■ Problem Statement

Traditional satellite crop monitoring relies on standard 3-channel RGB (Red, Green, Blue) or simple 4-channel Multispectral imagery (e.g., NDVI using Near-Infrared). These conventional models can only detect crop diseases **after** leaves turn yellow or brown (foliar chlorosis and necrosis). By the time visual symptoms appear to human eyes or standard 2D Convolutional Neural Networks (CNNs), the fungal infection has already destroyed the internal cell structures of the plants, making harvest losses unrecoverable.

Furthermore, standard 2D-CNN architectures are fundamentally incapable of processing the massive 3D depth of spectral band information required for early biochemical stress detection.

■ Real-World Use Case & Vision

An agricultural analyst monitors a 1,000-acre commercial farm via the **TerraSpectra** platform. The system ingests **Hyperspectral Satellite Data Cubes** capturing **200+ contiguous spectral bands of light** across the electromagnetic spectrum (from 400nm to 2500nm), far beyond human visual perception.

The backend **Hybrid 3D-CNN + Vision Transformer (ViT)** AI model analyzes subtle chemical changes in the plants' chlorophyll-a/b absorption, anthocyanin accumulation, and cellular water retention. The TerraSpectra GIS dashboard automatically highlights a specific 5-acre zone in glowing red, predicting a **fungal blight outbreak three weeks before any visible symptoms appear on the leaves**. This early warning allows farm managers to execute hyper-targeted, localized preventative treatment, saving 95% of pesticide costs and preserving crop yield.

■■ Core System Modules

- Geospatial Data Pipeline (Python, Rasterio, GDAL, HDF5, NetCDF4):** Ingests, calibrates, and normalizes multi-gigabyte hyperspectral data cubes (from NASA Hyperion, ESA Sentinel-6, or AVIRIS airborne sensors).
- 3D-CNN & ViT Hybrid AI Model (PyTorch):** A complex neural network utilizing 3D convolutions to capture spatial features alongside a Vision Transformer (ViT) self-attention engine to process deep inter-spectral band correlations.
- High-Throughput Inference API (FastAPI):** REST API capable of GPU-accelerated image chunking, sliding-window tiling, and real-time prediction raster generation.
- Interactive GIS Dashboard (React 18, Deck.gl, Mapbox GL JS):** A professional 3D map interface rendering predicted disease heatmaps overlaid on satellite topography with interactive spectral analytics.

SECTION 2: Week 1 Master Plan & Deep-Dive Schedule

Week	ML Engineering & Data Pipeline (PyTorch, Rasterio)	Frontend & GIS (React, Deck.gl)
Week 1	Data Preparation: Use Rasterio to parse mock hyperspectral data cubes (HDF5/GeoTIFF formats). Perform dimensionality reduction (PCA) on the 200+ spectral bands.	Map Scaffolding: Build React app. Integrate Deck.gl and Mapbox to render the base geographical farm layers.

PHASE 1: Deep Research & Learning Matrix (Prerequisites & Theoretical Foundations)

Before writing code, all 4 team members must master the following core domain concepts, mathematical formulas, file formats, and GIS concepts.

1.1 Hyperspectral Remote Sensing Physics

- Electromagnetic Spectrum Range:**
- Visible (VIS):** $400\text{ nm} - 700\text{ nm}$ (Pigment absorption: Chlorophyll-a at 675 nm , Chlorophyll-b at 650 nm).
- Near-Infrared (NIR):** $700\text{ nm} - 1100\text{ nm}$ (Cellular structure reflection: High reflectance in healthy leaf spongy mesophyll).
- Short-Wave Infrared 1 (SWIR-1):** $1100\text{ nm} - 1400\text{ nm}$ (Foliar moisture & protein content).
- Short-Wave Infrared 2 (SWIR-2):** $1400\text{ nm} - 2500\text{ nm}$ (Lignin, cellulose, and severe water stress absorption troughs at 1450 nm and 1940 nm).
- Early Disease Detection Signature (The "Red Edge Shift"):**
- When fungal blight attacks a plant, cell wall damage causes a leftward shift in the **Red Edge region** ($680\text{ nm} - 750\text{ nm}$) toward shorter wavelengths weeks before chlorophyll completely breaks down.

1.2 Data File Formats & GIS Coordinate Systems

- **HDF5 (.h5 / .hdf5):** Hierarchical Data Format 5. Stores multi-dimensional arrays (tensors) with metadata attributes. Structure: File -> Group -> Dataset (Height, Width, Bands).
- **GeoTIFF (.tif / .tiff):** Raster image file formatted with geospatial tags (GeoKeys). Contains affine transformation parameters mapping pixel (x, y) to real-world GPS coordinates (Longitude, Latitude).
- **Coordinate Reference Systems (CRS):**
- **EPSG:4326 (WGS 84):** Geographic coordinate system in decimal degrees (Latitude, Longitude). Standard for GPS and API payloads.
- **EPSG:3857 (Web Mercator):** Projected coordinate system in meters. Standard for web mapping engines (Mapbox, Google Maps, OpenStreetMap).

1.3 Mathematics of Dimensionality Reduction (PCA)

Hyperspectral data cubes possess extreme spectral redundancy (adjacent bands like 550nm and 551nm are highly correlated). Storing 200+ bands in RAM for neural network training causes the **Curse of Dimensionality** and memory crashes.

PCA Mathematical Derivation:

Reshape 3D Cube to 2D Matrix: Given a Data Cube $C \in \mathbb{R}^{H \times W \times B}$ where $H = \text{Height}$, $W = \text{Width}$, $B = 224 \text{ bands}$: $X = \text{Reshape}(C) \in \mathbb{R}^{N \times B} \quad \text{where } N = H \times W$

Mean Centering: $\mu_j = \frac{1}{N} \sum_{i=1}^N X_{[i, j]} \quad \text{for } j \in [1, B]$
 $\bar{X} = X - \mu$

Covariance Matrix Calculation: $\Sigma = \frac{1}{N-1} \bar{X}^T \bar{X} \in \mathbb{R}^{B \times B}$

Eigenvalue Decomposition: $\Sigma V = V \Lambda$ Where $V = [v_1, v_2, \dots, v_B]$ are eigenvectors, and $\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_B)$ are eigenvalues sorted in descending order ($\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_B$).

Cumulative Explained Variance Ratio (EVR): Choose top k principal components such that:
 $\text{EVR}(k) = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^B \lambda_j} \geq 0.985 \quad (98.5\% \text{ variance retained})$
Typically, $k = 16$ to 32 components retain $>98.5\%$ of total spectral information while reducing memory by **85%**.

Dimensionality Projection: $Z = \bar{X} V_k \in \mathbb{R}^{N \times k} \xrightarrow{\text{Reshape}} C_{\text{reduced}} \in \mathbb{R}^{H \times W \times k}$

■ PHASE 2: Execution & Implementation Guide (Step-by-Step Production Code)

■ Task 1: Synthetic Hyperspectral Data Cube Generator Script

Target File: `scripts/generate_mock_data.py` **Assigned Member:** Member 2 (Geospatial Data Engineer)

This script generates a realistic 224-band HDF5 hyperspectral dataset containing simulated healthy vegetation, soil, water, and an **early-stage fungal infection zone** with authentic spectral reflectance curves across 400nm - 2500nm\$.

```
import os
import h5py
import numpy as np
import rasterio
from rasterio.transform import from_bounds

def generate_synthetic_hyperspectral_cube(
    output_h5_path: str,
    output_tif_path: str,
    height: int = 512,
    width: int = 512,
    num_bands: int = 224
):
    """
    Generates a synthetic 224-band Hyperspectral Data Cube simulating an agricultural farm
    with healthy crops, soil, water bodies, and an early-stage fungal blight outbreak.
    """
    os.makedirs(os.path.dirname(output_h5_path), exist_ok=True)
    print(f"Generating synthetic hyperspectral data cube ({height}x{width}x{num_bands})...")

    # 1. Generate Wavelengths (400nm to 2500nm)
    wavelengths = np.linspace(400, 2500, num_bands)

    # 2. Base Reflectance Curves
    veg_curve = np.zeros(num_bands)
    for i, w in enumerate(wavelengths):
        if w < 500:
            veg_curve[i] = 0.05 + 0.02 * np.random.randn() # Blue absorption
        elif 500 <= w < 600:
            veg_curve[i] = 0.15 + 0.03 * np.exp(-((w - 550)**2)/800) # Green reflection peak
        elif 600 <= w < 680:
            veg_curve[i] = 0.04 + 0.01 * np.random.randn() # Red absorption
        elif 680 <= w < 750:
            veg_curve[i] = 0.05 + 0.45 * (w - 680) / 70 # Red Edge steep incline
        elif 750 <= w < 1300:
            veg_curve[i] = 0.50 + 0.03 * np.random.randn() # NIR plateau
        elif 1300 <= w < 1500:
            veg_curve[i] = 0.25 - 0.15 * np.exp(-((w - 1450)**2)/1000) # Water absorption
        else:
            veg_curve[i] = 0.20 + 0.05 * np.random.randn() # SWIR region

    infected_curve = veg_curve.copy()
    for i, w in enumerate(wavelengths):
        if 680 <= w < 750:
            infected_curve[i] *= 0.6 # Chlorophyll breakdown
        elif 750 <= w < 1300:
            infected_curve[i] *= 0.55 # Cell structure breakdown
        elif 1300 <= w < 1500:
            infected_curve[i] += 0.12 # Foliar water loss

    # 3. Create Spatial Grid
    cube = np.zeros((height, width, num_bands), dtype=np.float32)
    labels = np.zeros((height, width), dtype=np.int32) # 0: Healthy, 1: Infected Blight Zone
```

```

    for b in range(num_bands):
        cube[:, :, b] = veg_curve[b] + np.random.normal(0, 0.02, (height, width))

    cy, cx, radius = 250, 300, 60
    y_grid, x_grid = np.ogrid[:height, :width]
    dist_from_center = np.sqrt((x_grid - cx)**2 + (y_grid - cy)**2)
    infection_mask = dist_from_center <= radius

    for b in range(num_bands):
        cube[:, :, b][infection_mask] = infected_curve[b] + np.random.normal(0, 0.02, np.sum(infection_mask))
    labels[infection_mask] = 1

    cube = np.clip(cube, 0.0, 1.0)

    # 4. Save to HDF5 Format
    with h5py.File(output_h5_path, 'w') as h5f:
        h5f.create_dataset('hyperspectral_cube', data=cube, compression='gzip', chunks=(64, 64, num_bands))
        h5f.create_dataset('ground_truth_labels', data=labels, compression='gzip')
        h5f.create_dataset('wavelengths', data=wavelengths)
        h5f.attrs['location'] = 'Iowa Farm Zone Alpha'
        h5f.attrs['sensor'] = 'Synthetic Hyperion AVIRIS-NG'
        h5f.attrs['epsg'] = 4326
        h5f.attrs['bbox'] = [-93.62, 41.58, -93.60, 41.60]

    print(f"■ HDF5 Hyperspectral Data Cube saved to: {output_h5_path}")

    # 5. Save RGB Synthetic Proxy GeoTIFF using Rasterio
    r_idx = np.argmin(np.abs(wavelengths - 650))
    g_idx = np.argmin(np.abs(wavelengths - 550))
    b_idx = np.argmin(np.abs(wavelengths - 470))

    rgb_raster = np.stack([cube[:, :, r_idx], cube[:, :, g_idx], cube[:, :, b_idx]], axis=0)
    rgb_uint8 = (rgb_raster * 255).astype(np.uint8)

    transform = from_bounds(-93.62, 41.58, -93.60, 41.60, width, height)

    with rasterio.open(
        output_tif_path,
        'w',
        driver='GTiff',
        height=height,
        width=width,
        count=3,
        dtype=np.uint8,
        crs='EPSG:4326',
        transform=transform,
    ) as dst:
        dst.write(rgb_uint8)

    print(f"■ GeoTIFF RGB preview saved to: {output_tif_path}")

    if __name__ == "__main__":
        generate_synthetic_hyperspectral_cube(
            output_h5_path="data/mock_farm_datacube.h5",
            output_tif_path="data/mock_farm_preview.tif"
        )

```

■ Task 2: Rasterio Data Ingestion & PCA Reduction Pipeline

Target Files: - [backend/app/services/data_pipeline.py](#) - [backend/app/services/pca_reducer.py](#)

Assigned Member: Member 2 (Geospatial Data Engineer)

data_pipeline.py (Rasterio & HDF5 Data Loader):

```
import h5py
import numpy as np
import rasterio
from typing import Dict, Any, Tuple

class HyperspectralDataLoader:
    """
    Ingests 3D Hyperspectral Data Cubes from HDF5 and GeoTIFF formats,
    extracting spatial dimensions, spectral metadata, and coordinate reference systems.
    """

    @staticmethod
    def load_hdf5_cube(file_path: str) -> Tuple[np.ndarray, np.ndarray, Dict[str, Any]]:
        with h5py.File(file_path, 'r') as h5f:
            cube = h5f['hyperspectral_cube'][:]
            wavelengths = h5f['wavelengths'][:]
            metadata = {
                'sensor': h5f.attrs.get('sensor', 'Unknown'),
                'bbox': list(h5f.attrs.get('bbox', [-93.62, 41.58, -93.60, 41.60])),
                'epsg': int(h5f.attrs.get('epsg', 4326)),
                'shape': cube.shape
            }
        return cube, wavelengths, metadata

    @staticmethod
    def read_geotiff_bounds(geotiff_path: str) -> Dict[str, Any]:
        with rasterio.open(geotiff_path) as dataset:
            bounds = dataset.bounds
            crs = dataset.crs.to_string()
            transform = dataset.transform
            return {
                'bounds': [bounds.left, bounds.bottom, bounds.right, bounds.top],
                'crs': crs,
                'width': dataset.width,
                'height': dataset.height,
                'transform': transform
            }
```

pca_reducer.py (Dimensionality Reduction Module):

```
import numpy as np
from sklearn.decomposition import PCA
from typing import Tuple, Dict

class SpectralPCAReducer:
    """
    Applies Principal Component Analysis (PCA) to reduce 200+ hyperspectral bands
    to target N components while maintaining >98.5% cumulative explained variance.
    """
```

```

def __init__(self, n_components: int = 32):
    self.n_components = n_components
    self.pca = PCA(n_components=n_components)

def fit_transform_cube(self, cube: np.ndarray) -> Tuple[np.ndarray, Dict[str, Any]]:
    height, width, bands = cube.shape
    flattened_matrix = cube.reshape(-1, bands)
    reduced_matrix = self.pca.fit_transform(flattened_matrix)
    reduced_cube = reduced_matrix.reshape(height, width, self.n_components)

    explained_variance_ratio = self.pca.explained_variance_ratio_
    cumulative_variance = np.sum(explained_variance_ratio)

    stats = {
        'original_bands': bands,
        'reduced_components': self.n_components,
        'cumulative_explained_variance': float(cumulative_variance),
        'per_component_variance': explained_variance_ratio.tolist()
    }

    print(f"■ PCA Completed: {bands} bands -> {self.n_components} components.")
    print(f"■ Cumulative Retained Variance: {cumulative_variance * 100:.2f}%")

    return reduced_cube, stats

```

■ Task 3: React 18 + Deck.gl + Mapbox Map Scaffolding

Target Component: [frontend/src/components/MapContainer.tsx](#) **Assigned Member:** Member 4 (Frontend GIS Developer)

```

import React, { useState } from 'react';
import DeckGL from '@deck.gl/react/typed';
import { Map } from 'react-map-gl';
import { GeoJsonLayer } from '@deck.gl/layers/typed';
import { Activity } from 'lucide-react';

const MAPBOX_TOKEN = import.meta.env.VITE_MAPBOX_ACCESS_TOKEN || 'pk.eyJ1IjoibWFwYm94IiwiYSI6ImNpejY4M29iaiazA';

export const MapContainer: React.FC = () => {
    const [viewState, setViewState] = useState({
        longitude: -93.61,
        latitude: 41.59,
        zoom: 14.5,
        pitch: 45,
        bearing: 0
    });

    const farmBoundaryGeoJSON = {
        type: 'FeatureCollection',
        features: [
            {
                type: 'Feature',
                geometry: {
                    type: 'Polygon',
                    coordinates: [

```

```

        [-93.62, 41.58],
        [-93.60, 41.58],
        [-93.60, 41.60],
        [-93.62, 41.60],
        [-93.62, 41.58]
    ]
    ],
    },
    properties: { name: 'Iowa Farm Zone Alpha', acres: 1000 }
  }
]
};

const layers = [
  new GeoJsonLayer({
    id: 'farm-boundary',
    data: farmBoundaryGeoJSON,
    stroked: true,
    filled: true,
    getFillColor: [16, 185, 129, 40],
    getLineColor: [16, 185, 129, 255],
    getLineWidth: 3,
    lineWidthMinPixels: 2
  })
];

return (
  <div className="relative w-full h-screen bg-slate-950 overflow-hidden font-sans">
    <header className="absolute top-4 left-4 z-20 bg-slate-900/90 backdrop-blur-md border border-slate-800">
      <div className="p-2 bg-emerald-500/20 rounded-lg text-emerald-400">
        <Activity className="w-6 h-6" />
      </div>
      <div>
        <h1 className="text-lg font-bold tracking-wide text-emerald-400">TerraSpectra GIS</h1>
        <p className="text-xs text-slate-400">Hyperspectral Fungal Blight Early Warning System</p>
      </div>
    </header>

    <DeckGL
      viewState={viewState}
      onViewStateChange={(e: any) => setViewState(e.viewState)}
      controller={true}
      layers={layers}
    >
      <Map
        mapboxAccessToken={MAPBOX_TOKEN}
        mapStyle="mapbox://styles/mapbox/satellite-v9"
        reuseMaps
      />
    </DeckGL>

    <div className="absolute bottom-6 left-6 z-20 bg-slate-900/90 backdrop-blur-md border border-slate-800">
      <span className="flex items-center gap-1.5 text-emerald-400 font-semibold">
        <span className="w-2 h-2 rounded-full bg-emerald-500 animate-pulse"></span>
        224 Bands Ingested
      </span>
      <span className="text-slate-600">|</span>
      <span>Resolution: 1.0m/px</span>
      <span className="text-slate-600">|</span>
    </div>
  </div>
);

```



```
        <span>CRS: EPSG:4326</span>
    </div>
</div>
);
};
```

■ Task 4: FastAPI Core Backend Server

Target File: `backend/main.py` **Assigned Member:** Member 1 (System Integration & API Lead)

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import os
import sys

sys.path.append(os.path.dirname(os.path.abspath(__file__)))

from app.services.data_pipeline import HyperspectralDataLoader
from app.services.pca_reducer import SpectralPCAReducer

app = FastAPI(
    title="TerraSpectra Core API",
    description="Hyperspectral Data Cube Pipeline & AI Disease Forecasting Server",
    version="1.0.0"
)

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

class DataCubeRequest(BaseModel):
    file_path: str
    target_components: int = 32

@app.get("/health")
def health_check():
    return {
        "status": "online",
        "service": "TerraSpectra Core Engine",
        "gpu_available": False
    }

@app.post("/api/v1/pca/reduce")
def process_pca_reduction(payload: DataCubeRequest):
    if not os.path.exists(payload.file_path):
        raise HTTPException(status_code=404, detail=f"HDF5 file not found: {payload.file_path}")

    try:
        cube, wavelengths, metadata = HyperspectralDataLoader.load_hdf5_cube(payload.file_path)
        reducer = SpectralPCAReducer(n_components=payload.target_components)
        reduced_cube, stats = reducer.fit_transform_cube(cube)
```

```
|  
    return {  
        "status": "success",  
        "metadata": metadata,  
        "pca_statistics": stats,  
        "reduced_shape": reduced_cube.shape  
    }  
    except Exception as e:  
        raise HTTPException(status_code=500, detail=str(e))  
|  
if __name__ == "__main__":  
    import uvicorn  
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```